Programming Elective - I Java (CS405-PE)

B.Tech CSE IV Semester | Detailed Semester Exam Notes

Prepared for Rahul Yadav. These notes cover Servlet, JSP, Spring, Spring Boot, JPA/Hibernate, MVC and RESTful Web Services with theory, diagrams, tables, examples and exam-focused revision points.

Unit	Main Topics	Exam Focus
I	Servlet, JSP, session, filters, JSP elements	Lifecycle diagrams, web.xml, RequestDispatcher, implicit objec
II	Spring Framework and Spring Boot	IoC, DI, Bean lifecycle, scopes, annotations, auto-configuration
III	JPA/Hibernate	Entity mapping, primary keys, relationships, fetch types, JPQL,
IV	Spring Boot MVC	MVC layers, controllers, request handling, Thymeleaf, JSON, e
V	RESTful Web Services	REST architecture, HTTP methods, DTOs, content negotiation,

How to use these notes: first read every unit's lifecycle/architecture diagram, then memorize tables, then practice short code snippets. For long answers, write definition, diagram, working steps, advantages and one example.

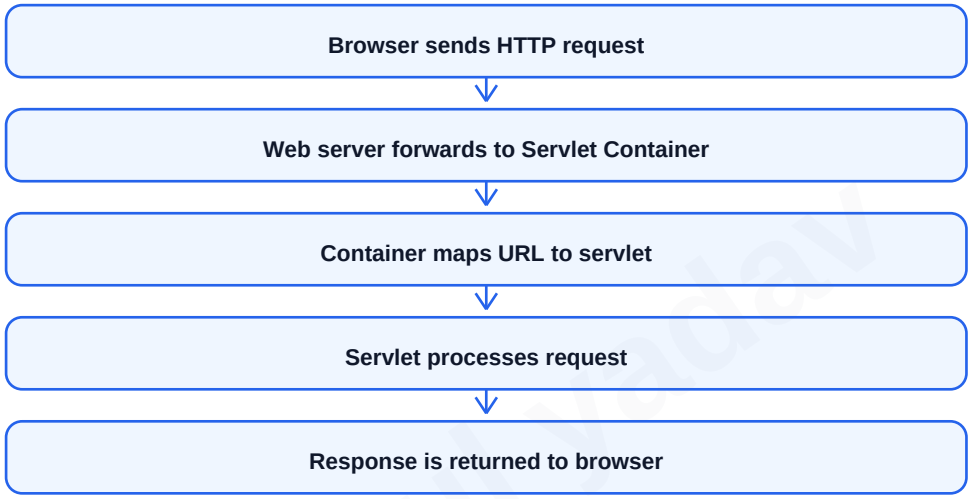
UNIT I: Servlet and JSP - Servlet Basics

A servlet is a Java server-side component that runs inside a servlet container such as Apache Tomcat. It receives client requests, processes them using Java logic and sends dynamic responses back to the browser. Servlets are commonly used in web applications where content depends on user input, database values or session state.

The servlet container manages servlet object creation, lifecycle methods, multithreading, request-response objects, deployment descriptors and security. A servlet normally extends HttpServlet and overrides doGet or doPost according to the request method.

In exam answers, mention that servlet is platform independent, secure, efficient and integrated with the Java EE web container.

- Servlets are faster than old CGI because a new process is not created for every request.
- Servlets can access request parameters, headers, cookies, sessions and context information.
- Servlets usually generate HTML directly or forward data to JSP for presentation.
- A web application is deployed as a WAR file or exploded folder inside the server.



Term	Meaning
Servlet	Java class that handles web requests
Servlet Container	Runtime that manages servlet lifecycle
HttpServletRequest	Carries client request data
HttpServletResponse	Used to create response
WAR	Web Application Archive for deployment

Servlet Lifecycle and Environment Setup

Servlet lifecycle means the sequence of steps through which a servlet object passes from creation to destruction. The container loads the servlet class, creates its object, calls init once, calls service for every request and finally calls destroy before removing it from memory.

The init method is used for one-time initialization such as reading configuration or opening resources. The service method identifies the HTTP method and dispatches to doGet, doPost, doPut or doDelete. The destroy method releases resources such as database connections or background tasks.

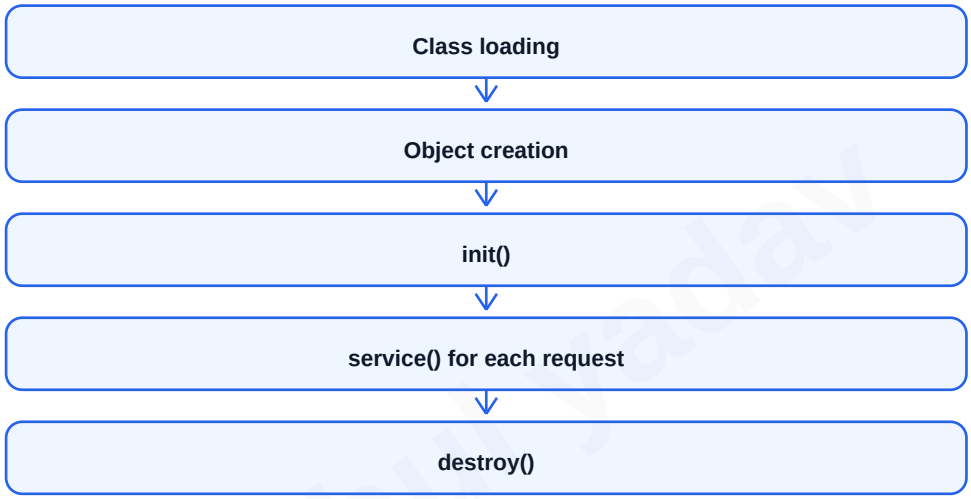
To set up a servlet environment, install JDK, install Tomcat, create a Dynamic Web Project or Maven web project, add servlet API dependency, create servlet class, configure URL mapping and run the project on server.

- init() is called only once in servlet life.

- service() is called for every request and may run in multiple threads.

- destroy() is called before servlet removal or server shutdown.

- Thread safety is important because one servlet object can serve many users.




```
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws IOException {
        res.setContentType("text/html");
        res.getWriter().println("<h2>Hello Servlet</h2>");
    }
}
```

Servlet Configuration, Context and RequestDispatcher

Servlet configuration can be done through web.xml or annotations. In web.xml, servlet-name, servlet-class and url-pattern are defined. Modern projects commonly use @WebServlet annotation because it reduces XML configuration.

ServletConfig stores configuration information specific to one servlet. ServletContext stores application-wide information shared by all servlets in the same web application. ServletContext can also be used to read global init parameters and share attributes.

RequestDispatcher is used to forward a request to another resource or include output of another resource. In forward, control is transferred to another servlet/JSP and browser URL usually remains same. In include, output of another resource is included in current response.

- ServletConfig scope is servlet-specific.
- ServletContext scope is application-wide.
- forward() is used when another resource should generate final response.
- include() is used to combine partial output from another resource.

Feature	ServletConfig	ServletContext
Scope	One servlet	Whole web application
Object count	One per servlet	One per web application
Use	Servlet init parameters	Global data, application attributes
Access	getServletConfig()	getServletContext()

```
RequestDispatcher rd = request.getRequestDispatcher("home.jsp");
rd.forward(request, response);

// include example
request.getRequestDispatcher("header.jsp").include(request, response);
```

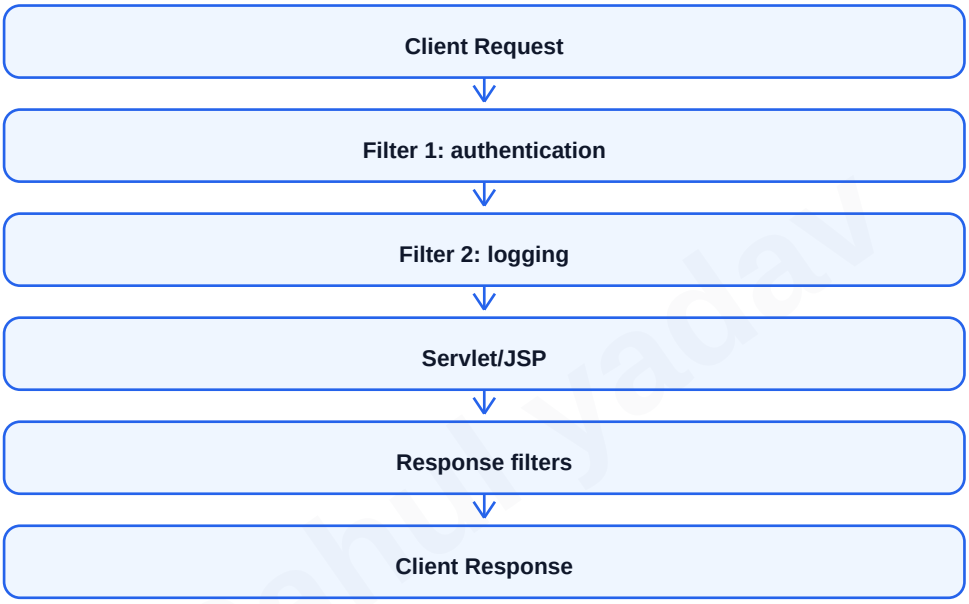
Session Management and Servlet Filters

HTTP is stateless, meaning each request is independent. Session management is the technique of maintaining user-specific data across multiple requests. In Java web applications, session management can be done using cookies, URL rewriting, hidden form fields and HttpSession.

HttpSession is the most common technique. The server creates a session object and sends a session id to the browser, usually through JSESSIONID cookie. User data such as login id or cart items can be stored as session attributes.

A filter is a reusable component that intercepts requests and responses before they reach a servlet or after servlet processing. Filters are used for authentication, logging, compression, input validation, encoding and security checks.

- Cookies store small data on client side.
- URL rewriting appends session id in URL when cookies are disabled.
- HttpSession stores data on server side.
- Filter chain allows multiple filters to run in defined order.



Technique	Storage	Main Use
Cookie	Client browser	Remember user/session id
Hidden Field	HTML form	Carry value through forms
URL Rewriting	URL	Session tracking without cookies
HttpSession	Server	Login data, cart data

JSP Introduction, Lifecycle and Elements

JSP stands for JavaServer Pages. It is a server-side technology used to create dynamic web pages by combining HTML with Java-based dynamic content. JSP is mainly used as the view layer, while business logic should be kept in servlet, service or bean classes.

A JSP page is translated into a servlet by the container. Then it is compiled, loaded, initialized and executed. Because JSP ultimately becomes a servlet, it follows a lifecycle similar to servlet. The important lifecycle methods are jspInit(), _jspService() and jspDestroy().

JSP scripting elements include declarations, scriptlets and expressions. Declarations define variables or methods at servlet class level. Scriptlets contain Java statements inside service method. Expressions print values into response.

- Declaration syntax: <%! declaration %>
- Scriptlet syntax: <% Java statements %>
- Expression syntax: <%= expression %>
- Modern applications avoid heavy Java code inside JSP and use JSTL/EL instead.


```
<%! int count = 0; %>
<% count++; %>
Current Count: <%= count %>
```

JSP Directives, Implicit Objects, JavaBeans and Action Tags

JSP directives provide instructions to the JSP container. The page directive defines page-level settings such as language, import, contentType and errorPage. The include directive includes a static file at translation time. The taglib directive declares a custom tag library such as JSTL.

JSP implicit objects are predefined objects available directly in JSP. They reduce boilerplate and help access request, response, session, application and page information. Common implicit objects are request, response, session, application, out, config, page, pageContext and exception.

JavaBeans are reusable Java classes with private properties, public getters/setters and no-argument constructor. JSP action tags such as jsp:useBean, jsp:setProperty and jsp:getProperty are used to work with beans.

- Use page directive for imports and content type.
- Use include directive for common header/footer.
- Use request object for form data.
- Use session object for user-specific data.
- Use JavaBean to separate data from presentation.

Implicit Object	Type/Meaning	Common Use
request	HttpServletRequest	Read form data and attributes
response	HttpServletResponse	Set response headers
session	HttpSession	Store user data
application	ServletContext	Application scope data
out	JspWriter	Print response
exception	Throwable	Available on error page

```
<jsp:useBean id="student" class="com.demo.Student" scope="request" />
<jsp:setProperty name="student" property="name" value="Rahul" />
Name: <jsp:getProperty name="student" property="name" />
```

UNIT II: Spring and Spring Boot Introduction

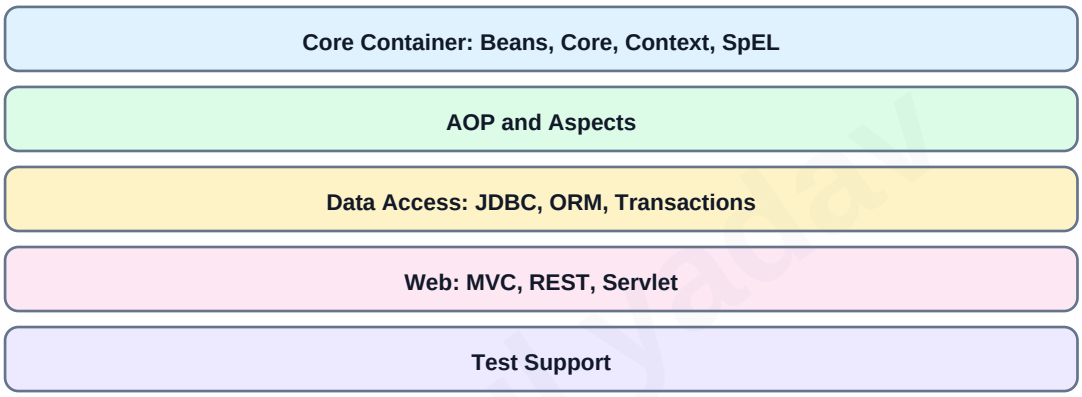
Spring Framework is a lightweight Java framework used to build enterprise applications. Its main goal is to simplify Java development through Inversion of Control, Dependency Injection, aspect-oriented programming, data access support, web MVC and transaction management.

Spring is modular. A developer can use only required modules such as Core Container, AOP, Data Access, Web MVC and Test. The heart of Spring is the IoC container, which creates objects, injects dependencies and manages bean lifecycle.

Spring Boot is built on top of Spring. It reduces configuration by providing starters, auto-configuration, embedded servers and production-ready features. It is widely used for microservices and REST APIs.

- Spring reduces tight coupling by injecting dependencies.
- Spring Boot reduces boilerplate setup and XML configuration.
- Embedded Tomcat allows running application as a normal Java program.
- Starter dependencies collect commonly used libraries for a feature.

Spring Framework Architecture



Spring	Spring Boot
Requires more manual configuration	Uses auto-configuration
External server often needed	Embedded server included
Dependency setup is manual	Starter dependencies available
Good for flexible framework-level control	Good for fast production-ready apps

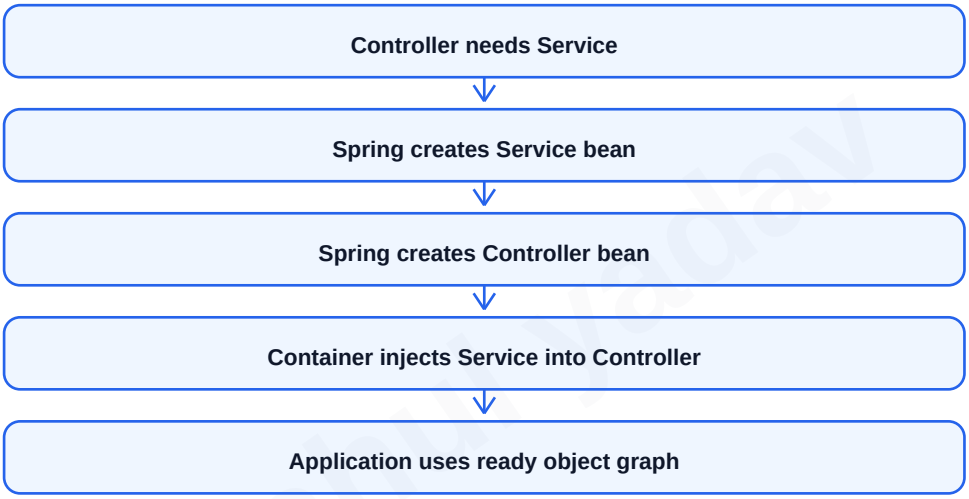
Inversion of Control and Dependency Injection

Inversion of Control means object creation and dependency management are transferred from application code to the Spring container. Instead of a class creating its required objects using new keyword, the container creates and supplies required dependencies.

Dependency Injection is the practical technique used to implement IoC. Dependencies can be injected through constructor, setter or field. Constructor injection is usually preferred because it makes required dependencies clear and supports immutable design.

DI improves testability because mock dependencies can be supplied during unit testing. It also improves maintainability because implementations can be changed without changing dependent class code.

- IoC is the principle; DI is the implementation technique.
- Constructor injection is preferred for mandatory dependencies.
- Setter injection is useful for optional dependencies.
- Field injection is simple but less test-friendly.
- DI makes classes loosely coupled.




```
@Service
class OrderService { }

@Controller
class OrderController {
    private final OrderService service;
    OrderController(OrderService service) {
        this.service = service;
    }
}
```

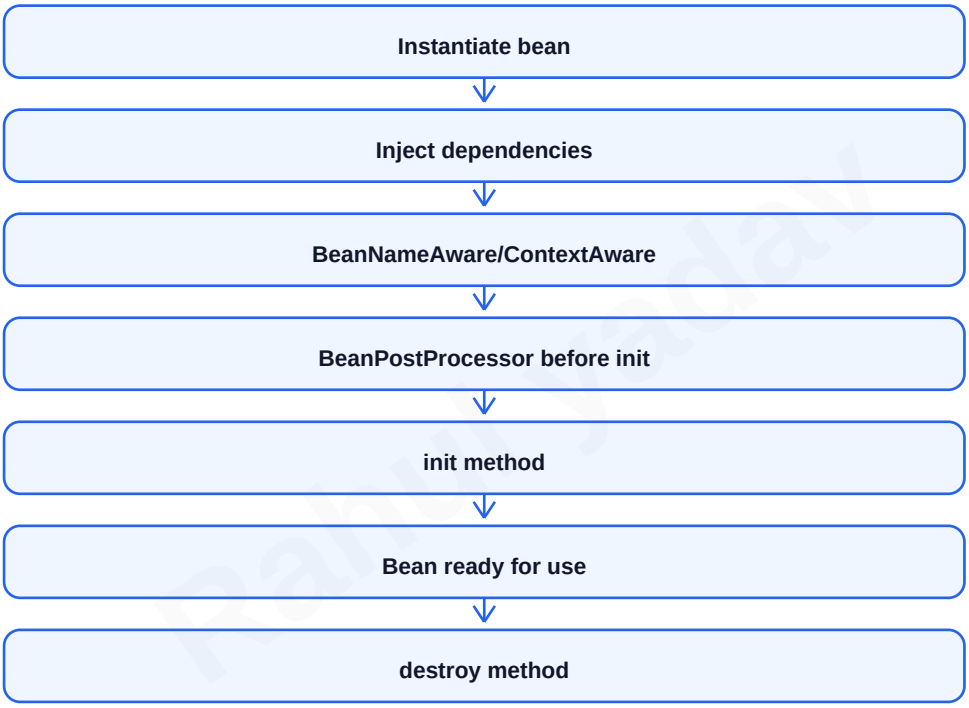
Spring Bean Lifecycle, Scopes and Configurations

A Spring bean is an object managed by the Spring IoC container. The container creates the bean, injects dependencies, calls initialization callbacks, makes it available for use and destroys it when context closes.

Bean configuration can be done using XML, Java configuration or annotations. Modern Spring applications mostly use annotation-based configuration with @Component, @Service, @Repository, @Controller and @Bean methods inside @Configuration classes.

Bean scope defines how many instances are created and how long they live. Singleton is default scope and creates one object per Spring container. Prototype creates a new object whenever requested. Web scopes include request, session and application.

- Bean lifecycle: instantiate, populate properties, aware callbacks, post-process, init, use, destroy.
- Singleton scope is default and memory efficient.
- Prototype scope is useful for stateful objects.
- @Bean is used when object creation needs custom logic.



Scope	Meaning	Typical Use
singleton	One bean object per container	Services, repositories
prototype	New object per request	Stateful helper
request	One object per HTTP request	Web request data
session	One object per HTTP session	User-specific state
application	One object per ServletContext	Shared web app data

Spring Core Annotations and ApplicationContext

Spring annotations reduce XML configuration and make components discoverable through classpath scanning. @Component is a generic stereotype. @Service marks business logic classes. @Repository marks data access classes and provides exception translation. @Controller marks MVC controllers. @RestController combines @Controller and @ResponseBody.

ApplicationContext is the advanced Spring container. It extends BeanFactory and provides enterprise features such as internationalization, event publishing, resource loading and annotation support. BeanFactory is simpler and lazy-loads beans by default.

In Spring Boot, the main application class usually has @SpringBootApplication, which combines @Configuration, @EnableAutoConfiguration and @ComponentScan.

- @Autowired injects dependencies.
- @Qualifier selects a bean when multiple beans of same type exist.
- @Value injects property values.
- @Configuration marks Java config class.
- @ComponentScan searches packages for components.

Annotation	Purpose
@Component	Generic Spring-managed component
@Service	Business/service layer class
@Repository	DAO/repository layer class
@Controller	MVC controller returning views
@RestController	REST controller returning body data
@SpringBootApplication	Main Boot annotation

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

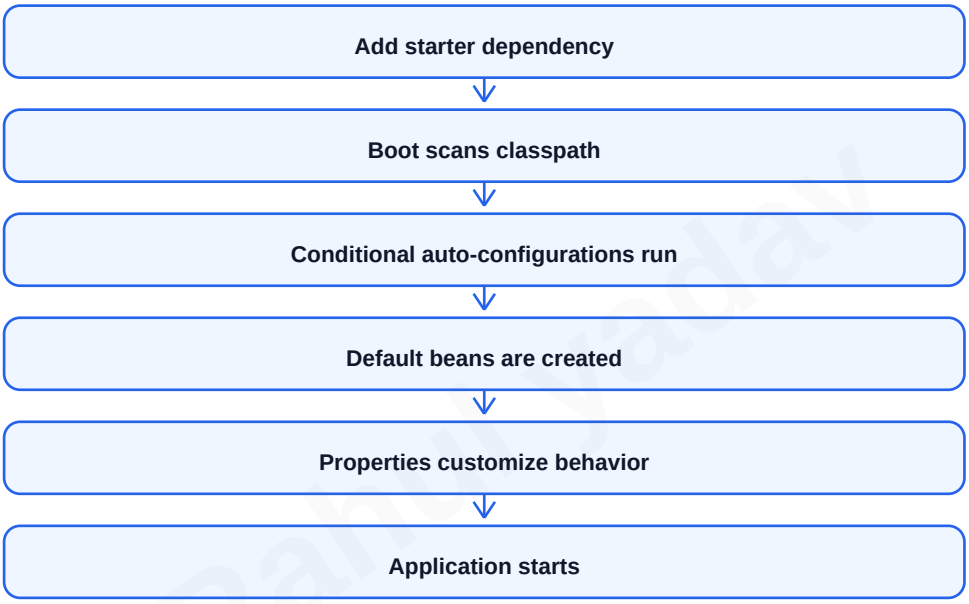
Spring Boot Starters and Auto-Configuration

Spring Boot starter is a dependency descriptor that brings a set of related libraries. For example, spring-boot-starter-web includes Spring MVC, Jackson, validation support and embedded Tomcat. This avoids remembering many separate dependencies.

Auto-configuration automatically configures beans based on classpath dependencies, properties and existing beans. If Spring Boot finds spring-webmvc and embedded Tomcat, it configures a web application. If it finds Spring Data JPA and a database driver, it configures JPA infrastructure.

Auto-configuration is conditional. It uses annotations like @ConditionalOnClass, @ConditionalOnMissingBean and @ConditionalOnProperty. Developers can override default beans by defining their own beans.

- Starters simplify dependency management.
- Auto-configuration follows convention over configuration.
- application.properties or application.yml customizes default behavior.
- Developer-defined beans can override Boot defaults.



Starter	Used For
spring-boot-starter-web	MVC and REST web apps
spring-boot-starter-data-jpa	JPA/Hibernate repositories
spring-boot-starter-thymeleaf	Server-side HTML views
spring-boot-starter-validation	Bean validation
spring-boot-starter-test	Testing support

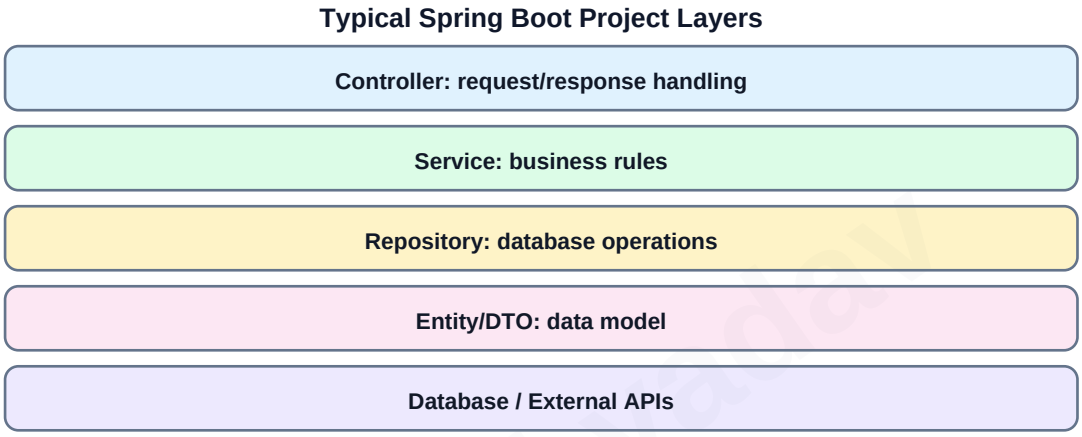
Creating a Spring Boot Application

A Spring Boot application can be created using Spring Initializr, IDE wizard or Maven/Gradle project setup. Select dependencies based on requirement, such as Spring Web, Thymeleaf, Data JPA, MySQL Driver and Validation.

The main class contains @SpringBootApplication and runs using SpringApplication.run. Controllers handle requests, services contain business logic, repositories interact with database and models/entities represent data.

In semester exams, write the project steps clearly: create project, add starter, create main class, create controller, configure properties, run application and test URL.

- Use layered package structure: controller, service, repository, entity, dto.
- Use application.properties for server port, datasource and JPA settings.
- Use Maven command mvn spring-boot:run or run main method from IDE.
- Keep controller thin and business logic in service layer.




```
@RestController
class HelloController {
    @GetMapping("/hello")
    String hello() {
        return "Hello Spring Boot";
    }
}
```

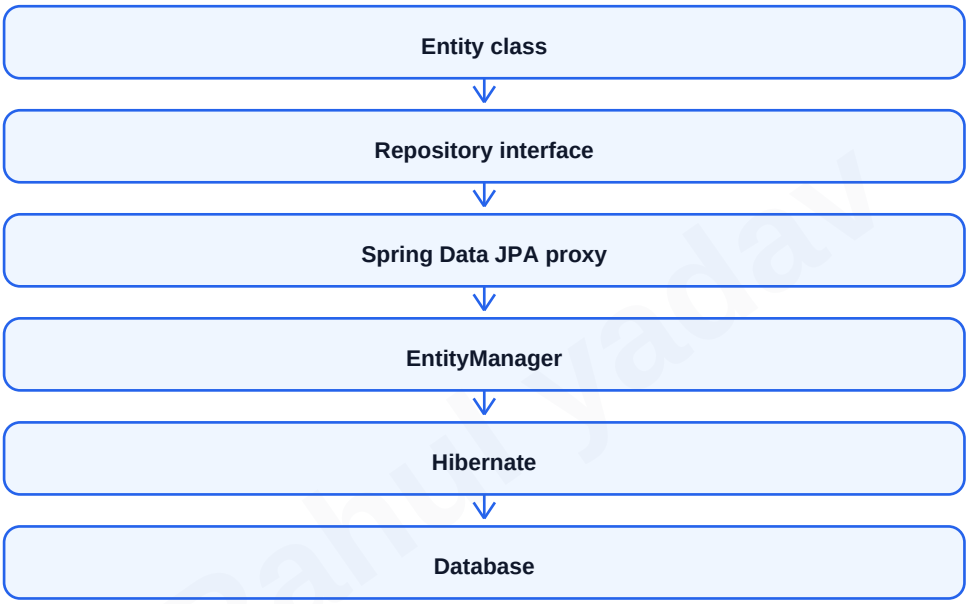
UNIT III: JPA/Hibernate Introduction and Setup

JPA stands for Java Persistence API. It is a specification for mapping Java objects to relational database tables. Hibernate is a popular implementation of JPA. In Spring Boot, Spring Data JPA simplifies repository creation and reduces boilerplate CRUD code.

Object Relational Mapping converts Java classes into database tables and Java objects into table rows. It helps developers work with objects instead of writing SQL for every simple operation.

To set up Spring Boot with JPA/Hibernate, add spring-boot-starter-data-jpa, add a database driver, configure datasource URL, username, password and JPA properties, create entity classes and create repository interfaces.

- JPA is a specification; Hibernate is an implementation.
- EntityManager is the JPA interface for persistence operations.
- Spring Data repositories provide ready-made CRUD methods.
- Hibernate generates SQL based on entity mapping.



Term	Meaning
ORM	Mapping objects to relational tables
Entity	Persistent Java class
Repository	Interface for data operations
EntityManager	JPA API for persistence context
Hibernate	JPA implementation

Entity Classes and Mapping Annotations

An entity class represents a database table. It is annotated with @Entity. The class usually has an id field annotated with @Id. @Table can specify table name. @Column can specify column name, length, nullable and uniqueness.

Entity classes should have a no-argument constructor because JPA creates objects using reflection. Fields are usually private with getters and setters. Entity should represent persistent state and should not contain heavy business logic.

Common annotations include @Entity, @Table, @Id, @GeneratedValue, @Column, @Transient, @Enumerated and @Lob.

- @Entity marks class as persistent.
- @Table customizes table name.
- @Id marks primary key.
- @GeneratedValue configures primary key generation.
- @Column customizes database column mapping.

Annotation	Purpose	Example
@Entity	Marks persistent class	@Entity
@Table	Defines table name	@Table(name="students")
@Id	Primary key field	@Id private Long id
@GeneratedValue	Auto id generation	GenerationType.IDENTITY
@Column	Column details	nullable=false

```
@Entity
@Table(name = "students")
class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 80)
    private String name;
}
```

Primary Keys and Generated Values

A primary key uniquely identifies each row in a database table. In JPA, the primary key field is annotated with @Id. Generated values are used when the database or persistence provider automatically creates primary key values.

GenerationType.IDENTITY uses database auto-increment column. GenerationType.SEQUENCE uses database sequence object. GenerationType.TABLE uses a separate table to store id values. GenerationType.AUTO lets the provider choose a suitable strategy.

Choice of strategy depends on database support and performance requirement. IDENTITY is simple and common with MySQL. SEQUENCE is common with PostgreSQL and Oracle.

- Primary key should be stable and unique.
- GeneratedValue reduces manual id assignment.
- IDENTITY is simple but may affect batch inserts.
- SEQUENCE can be efficient with allocation size.
- AUTO is portable but less explicit.

Strategy	Working	Suitable Database
IDENTITY	Uses auto-increment column	MySQL, SQL Server
SEQUENCE	Uses database sequence	PostgreSQL, Oracle
TABLE	Uses separate id table	Portable but slower
AUTO	Provider chooses strategy	General use

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE,
        generator = "student_seq")
@SequenceGenerator(name = "student_seq",
        sequenceName = "student_sequence",
        allocationSize = 1)
private Long id;
```

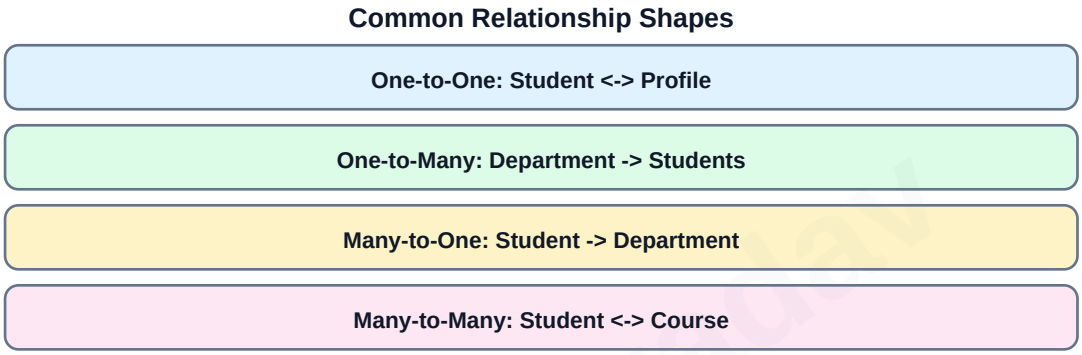
JPA Relationships and Mapping

Relationships describe how entity classes are connected. One-to-One means one record is related to exactly one record. One-to-Many means one parent has many child records. Many-to-One is the reverse side of One-to-Many. Many-to-Many means many records on both sides are connected through a join table.

In JPA, relationship annotations are @OneToOne, @OneToMany, @ManyToOne and @ManyToMany. The owning side contains the foreign key mapping. mappedBy is used on the inverse side to avoid duplicate relationship tables.

Cascade operations propagate persistence actions from parent to child. Orphan removal deletes child records removed from parent collection.

- Use @ManyToOne for foreign key from child to parent.
- Use mappedBy on inverse collection side.
- Use cascade carefully because it may delete related data.
- Use join table for many-to-many relationships.



Relationship	Annotation	Database Representation
One-to-One	@OneToOne	Unique foreign key
One-to-Many	@OneToMany	Foreign key in child table
Many-to-One	@ManyToOne	Child table has parent id
Many-to-Many	@ManyToMany	Join table

Fetch Types, JPQL and Native Queries

Fetch type decides when related data is loaded from database. Eager loading loads related data immediately with parent. Lazy loading loads related data only when accessed. Lazy loading improves performance when related data is not always required.

JPQL is Java Persistence Query Language. It queries entity classes and fields, not directly database tables and columns. Native SQL queries use actual database SQL and are useful for database-specific features or complex queries.

Query methods in Spring Data JPA can be derived from method names. For example, findByEmail or findByNameContaining automatically creates required query.

- Use lazy loading for large collections.
- Use eager loading only when related data is always needed.
- JPQL is database independent because it works with entities.
- Native queries are powerful but less portable.
- Use @Query for custom JPQL or native SQL.

Query Type	Works On	Advantage	Limitation
Derived Method	Repository method name	Very quick for simple queries	Hard for complex logic
JPQL	Entity and field names	Portable and object-oriented	Not all SQL features
Native SQL	Tables and columns	Full DB power	Database dependent

```
interface StudentRepository extends JpaRepository<Student, Long> {
    List<Student> findByNameContaining(String name);

    @Query("select s from Student s where s.email = :email")
    Optional<Student> findByEmail(String email);
}
```

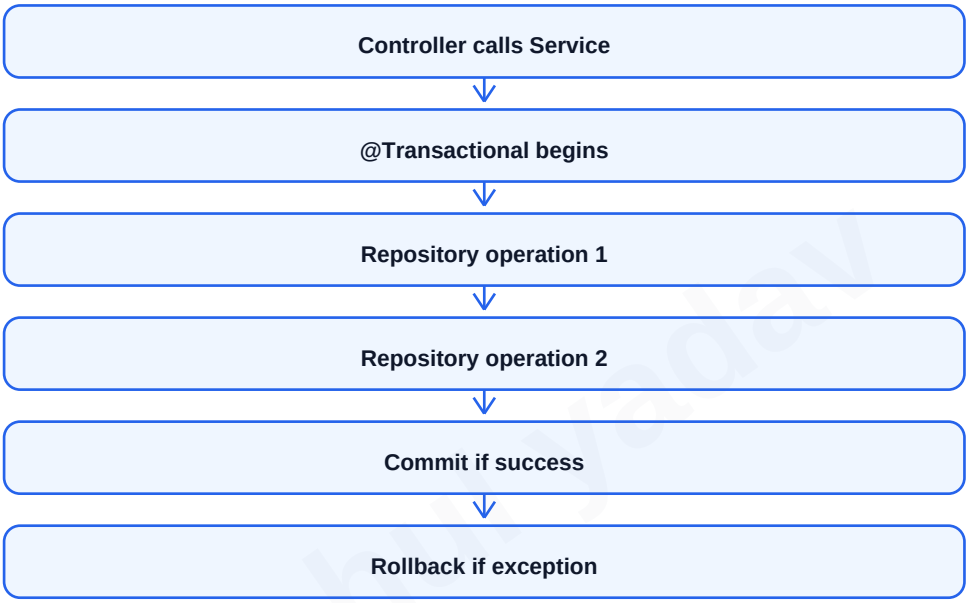
Spring Data JPA Repositories and Transactions

Spring Data JPA repository is an interface that provides common database operations without writing implementation class. JpaRepository provides methods such as save, findById, findAll, deleteById, count and pagination support.

Transaction management ensures that a group of database operations executes as a single unit. If all operations succeed, transaction commits. If an error occurs, transaction rolls back and database remains consistent.

@Transactional is used on service methods where multiple repository calls form one business operation. It is better to place transaction boundaries in service layer instead of controller layer.

- ACID properties are Atomicity, Consistency, Isolation and Durability.
- @Transactional rollback happens automatically for unchecked exceptions.
- Read-only transactions can improve performance for query operations.
- Repositories should focus on persistence; services should control business transactions.




```
@Service
class StudentService {
    @Transactional
    public Student create(Student student) {
        return repository.save(student);
    }
}
```

UNIT IV: Model-View-Controller Architecture

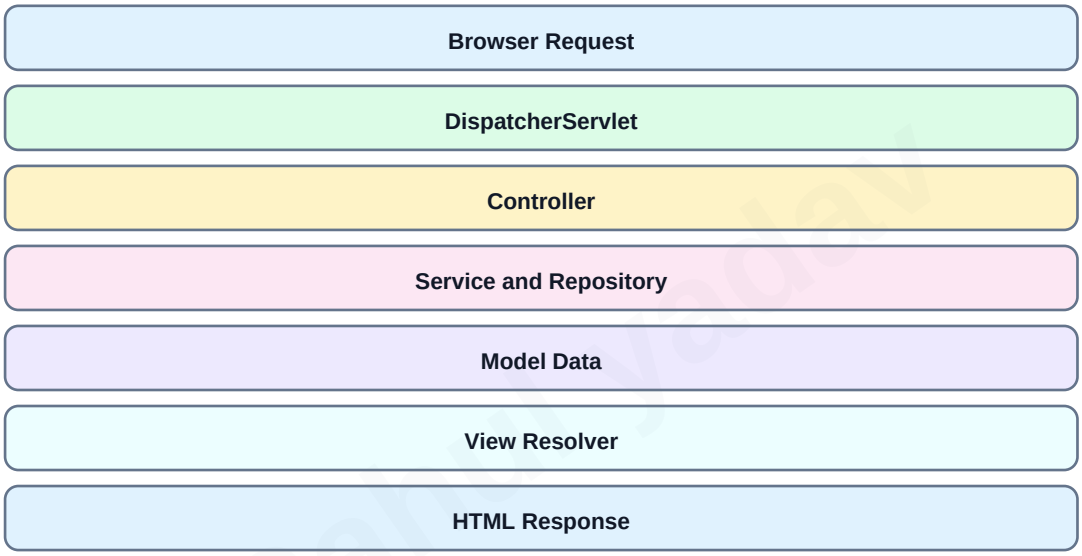
MVC is a software design pattern that separates an application into Model, View and Controller. Model represents data and business state. View displays data to user. Controller receives user request, calls service/model and selects the response.

Spring Boot MVC implements this pattern using DispatcherServlet as front controller. The DispatcherServlet receives all requests, finds the correct handler/controller, executes it, resolves the view and sends response.

MVC improves maintainability because presentation, request handling and business logic are separated. It also supports testing because controllers and services can be tested independently.

- Model stores data passed to view.
- View renders HTML using technologies like Thymeleaf or JSP.
- Controller maps URLs and handles request flow.
- DispatcherServlet is the front controller in Spring MVC.

Spring MVC Request Flow



Component	Responsibility
Model	Carries application data/state
View	Displays UI or template output
Controller	Handles request and selects response
DispatcherServlet	Central front controller
ViewResolver	Finds correct view template

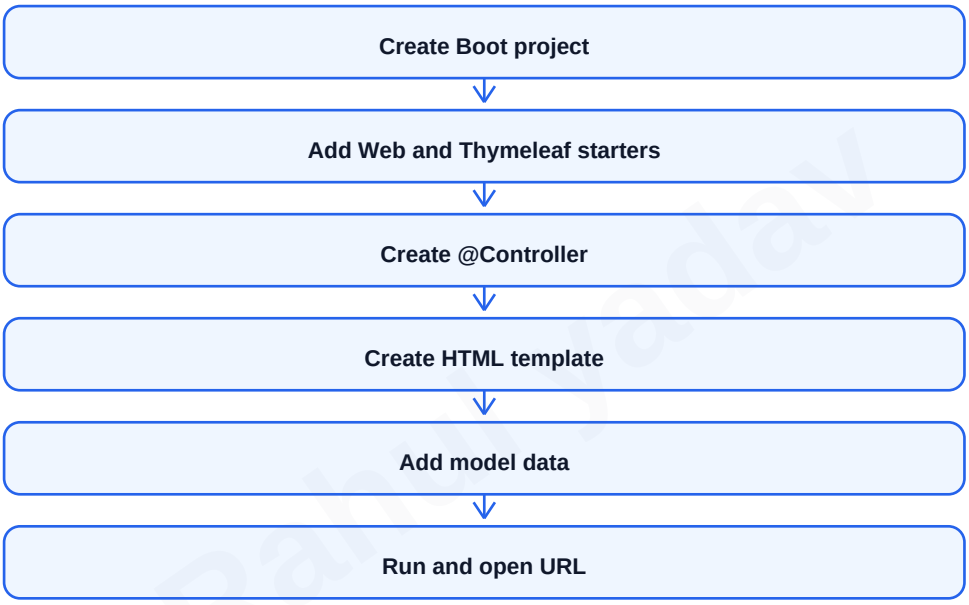
Setting up Spring Boot MVC Application

To create a Spring Boot MVC application, add spring-boot-starter-web and a template engine such as Thymeleaf. Create controller classes with @Controller. Place templates inside src/main/resources/templates and static files inside src/main/resources/static.

A controller method can return a view name. Data is passed to the view using Model object. Thymeleaf then renders dynamic HTML by reading model attributes.

A standard MVC project structure contains controller, service, repository, entity/model and dto packages. This structure keeps code clean for exam and practical implementation.

- Use @Controller when returning HTML views.
- Use Model addAttribute to pass values to page.
- Templates folder contains .html Thymeleaf views.
- Static folder contains CSS, JS and images.
- application.properties can configure server port and view behavior.




```
@Controller
class StudentPageController {
    @GetMapping("/students")
    String students(Model model) {
        model.addAttribute("title", "Student List");
        return "students";
    }
}
```

Spring MVC Annotations and Request Handling

Spring MVC provides annotations to map requests and bind request data. @RequestMapping is a general mapping annotation. @GetMapping, @PostMapping, @PutMapping and @DeleteMapping are shortcut annotations for HTTP methods.

@RequestParam reads query parameters or form fields. @PathVariable reads dynamic parts of URL path. @ModelAttribute binds form fields to an object. @RequestBody reads JSON request body and converts it into Java object using message converters.

Good controller design uses meaningful URL paths, correct HTTP methods and validation for incoming data.

- GET is used to read data.
- POST is used to create or submit data.
- PUT is used to update complete resource.
- DELETE is used to remove resource.
- Use validation annotations to check input.

Annotation	Purpose	Example
@GetMapping	Handle GET request	@GetMapping("/users")
@PostMapping	Handle POST request	@PostMapping("/users")
@RequestParam	Read query/form parameter	?name=Rahul
@PathVariable	Read URI variable	/users/5
@RequestBody	Read JSON body	DTO object
@ModelAttribute	Bind form object	Student form

```
@GetMapping("/students/{id}")
String getStudent(@PathVariable Long id,
                  @RequestParam(defaultValue = "false") boolean details) {
    return "student";
}
```

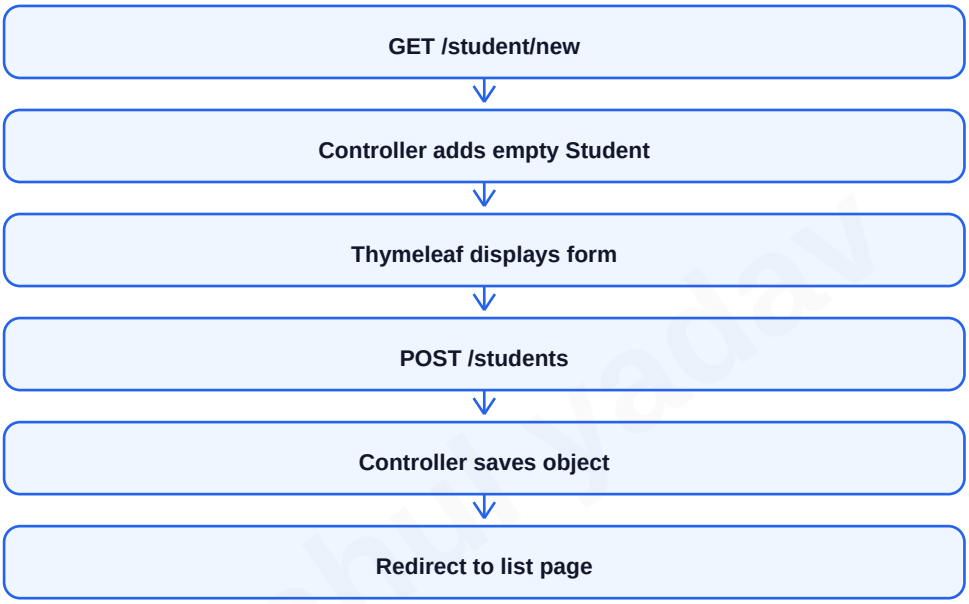
Model Attributes, Forms and Thymeleaf Integration

Model attributes are key-value pairs passed from controller to view. In Thymeleaf, expressions read these values and generate dynamic HTML. For example, th:text displays text, th:each loops over a collection and th:field binds form input to object property.

Form handling usually has two methods: one GET method to display empty form and one POST method to process submitted form. Submitted data can be bound using @ModelAttribute.

Thymeleaf is popular in Spring Boot because it works naturally with HTML templates and supports server-side rendering.

- Use model.addAttribute for page data.
- Use th:text for displaying dynamic text.
- Use th:each for loops.
- Use th:object and th:field for forms.
- Use redirect after POST to avoid duplicate form submission.




```
@PostMapping("/students")
String save(@ModelAttribute Student student) {
    service.save(student);
    return "redirect:/students";
}

<!-- Thymeleaf -->
<input th:field="*{name}" />
```

REST Controllers, JSON and Jackson

Spring Boot can return JSON responses using @RestController. It combines @Controller and @ResponseBody, so returned objects are written directly in HTTP response body. Jackson library converts Java objects to JSON and JSON to Java objects.

DTOs are recommended for API responses because they hide internal entity structure and allow clean input/output design. Controllers receive DTOs, services perform business logic and repositories persist entities.

JSON response is common for frontend frameworks, mobile apps and microservices. Spring Boot automatically configures Jackson when web starter is present.

- @RestController returns body data instead of view name.
- @RequestBody converts JSON request into Java object.
- ResponseEntity can set status code and headers.
- DTO prevents exposing unnecessary entity fields.
- Jackson supports serialization and deserialization.

Concept	Meaning
Serialization	Java object to JSON
Deserialization	JSON to Java object
DTO	Data Transfer Object for request/response
ResponseEntity	Response body plus status/header
Jackson	JSON mapper used by Spring Boot

```
@RestController
@RequestMapping("/api/students")
class StudentRestController {
    @GetMapping("/{id}")
    ResponseEntity<StudentDto> get(@PathVariable Long id) {
        return ResponseEntity.ok(service.findDto(id));
    }
}
```

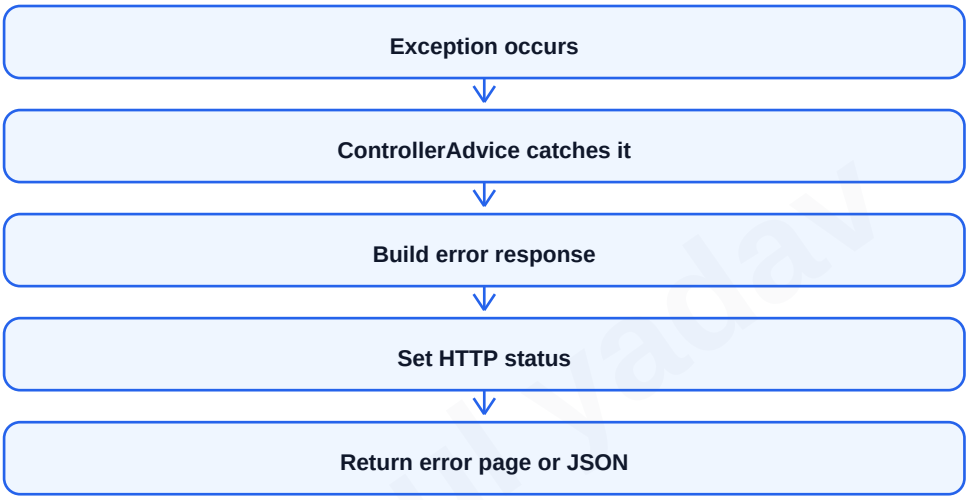
Exception Handling in Spring Boot MVC

Exception handling is important because errors should be returned in a clear and controlled format. In MVC pages, errors may show a user-friendly page. In REST APIs, errors should return proper HTTP status code and JSON error body.

@ExceptionHandler handles exceptions inside a controller. @ControllerAdvice or @RestControllerAdvice provides global exception handling for all controllers. ResponseStatusException can also be used to throw specific HTTP status errors.

Good error handling improves user experience, API reliability and debugging. Avoid exposing stack trace or sensitive internal details to users.

- Use 404 for resource not found.
- Use 400 for validation or bad request.
- Use 500 for server errors.
- Use global advice for common exceptions.
- Return consistent error response structure.




```
@RestControllerAdvice
class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    ResponseEntity<Map<String, String>> notFound(Exception ex) {
        return ResponseEntity.status(404)
                .body(Map.of("error", ex.getMessage()));
    }
}
```

UNIT V: RESTful Web Services and REST Architecture

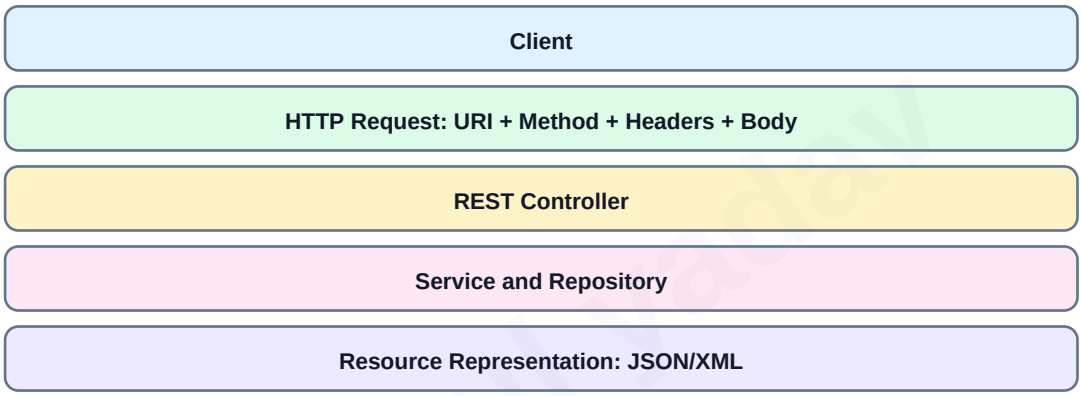
REST stands for Representational State Transfer. It is an architectural style for designing networked applications. A RESTful web service exposes resources through URIs and uses HTTP methods to perform operations on those resources.

A resource is any object or data item that can be identified, such as student, course or order. URI identifies the resource, for example /api/students/10. Representation is the format in which resource data is transferred, such as JSON or XML.

REST is stateless. Each request must contain all information required to process it. The server should not depend on previous request context except through explicitly provided tokens or data.

- Resource is identified by URI.
- HTTP methods define operation.
- JSON is common representation.
- Statelessness improves scalability.
- REST APIs should use meaningful status codes.

REST Architecture Concepts



REST Constraint	Meaning
Client-Server	UI and server are separated
Stateless	Each request is independent
Cacheable	Responses can define cache behavior
Uniform Interface	Consistent resource access
Layered System	Intermediaries such as gateway/proxy allowed

HTTP Methods, Status Codes and URI Design

REST APIs use HTTP methods according to operation type. GET reads resource, POST creates resource, PUT updates complete resource, PATCH updates part of resource and DELETE removes resource.

Status codes communicate result of request. 200 means success, 201 means created, 204 means success with no content, 400 means bad request, 401 means unauthenticated, 403 means forbidden, 404 means not found and 500 means server error.

URI design should be noun-based and resource-oriented. Use plural nouns such as /students and nested resources only when there is a real parent-child relationship.

- Use GET /students to list students.
- Use GET /students/{id} to read one student.
- Use POST /students to create.
- Use PUT /students/{id} to update.
- Use DELETE /students/{id} to delete.

Method	Operation	Example	Usual Status
GET	Read	GET /api/students/1	200 OK
POST	Create	POST /api/students	201 Created
PUT	Replace/update	PUT /api/students/1	200 OK
PATCH	Partial update	PATCH /api/students/1	200 OK
DELETE	Remove	DELETE /api/students/1	204 No Content

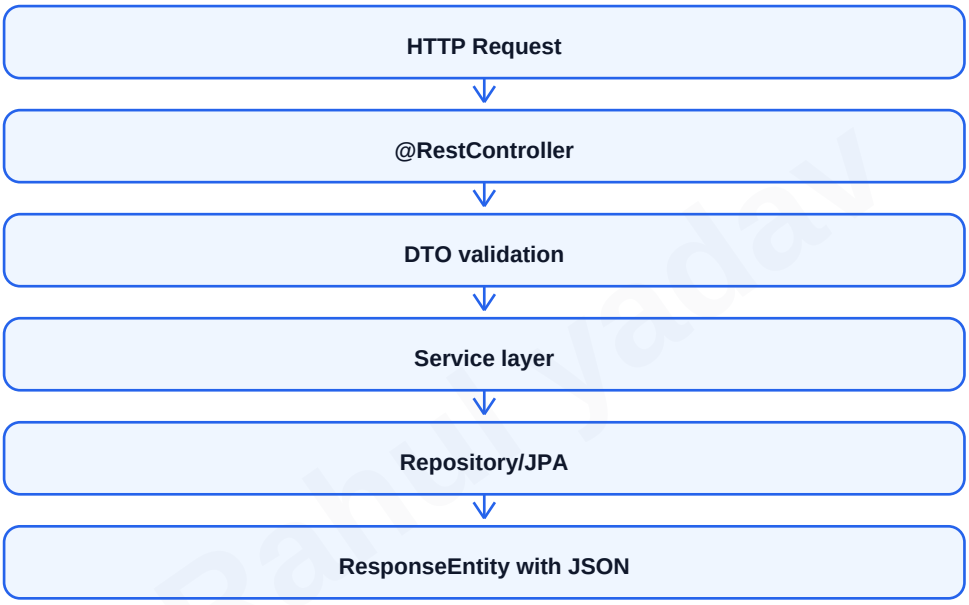
Creating REST Controllers with Spring Boot

In Spring Boot, REST controllers are created using @RestController. The base path is defined using @RequestMapping. Handler methods use @GetMapping, @PostMapping, @PutMapping and @DeleteMapping. Request and response bodies are commonly represented using DTO classes.

A clean REST controller does not contain database logic directly. It calls service layer methods. Service layer validates business rules and uses repository for persistence. This separation makes testing and maintenance easier.

ResponseEntity is useful for returning correct HTTP status code, headers and body. For create operation, return 201 Created with created object or location header.

- Keep controllers thin.
- Use DTOs for API input/output.
- Use @Valid for validation.
- Use ResponseEntity for status control.
- Use service layer for business logic.




```
@PostMapping
ResponseEntity<StudentDto> create(@Valid @RequestBody StudentDto dto) {
    StudentDto saved = service.create(dto);
    return ResponseEntity.status(HttpStatus.CREATED).body(saved);
}
```

Request and Response Entities, DTOs and Validation

Request entity represents data coming from client. Response entity represents data sent back to client. DTOs are used to shape request and response without exposing database entities. This is important for security, versioning and clean API design.

Validation ensures client sends correct data. Spring Boot supports Bean Validation using annotations like @NotBlank, @Email, @Size, @Min and @Max. @Valid triggers validation in controller method.

Mapping between entity and DTO can be manual or done using libraries. In exams, manual mapping is easy to explain.

- DTO avoids exposing passwords or internal ids unnecessarily.
- Request DTO may differ from response DTO.
- Validation errors should return 400 Bad Request.
- Use meaningful field messages.
- Use DTOs for stable API contracts.

Validation Annotation	Meaning	Example
@NotBlank	String must not be blank	name
@Email	Must be valid email	email
@Size	Length/collection size limit	password length
@Min	Minimum numeric value	age
@Max	Maximum numeric value	marks

```
class StudentRequest {
    @NotBlank
    private String name;

    @Email
    private String email;
}
```

Path Variables, Query Parameters and Content Negotiation

Path variables identify a specific resource in URI path, for example /students/5. Query parameters are used for filtering, searching, sorting and pagination, for example /students?page=0&size;=10&sort;=name.

Content negotiation means selecting response format based on request headers or URL configuration. A client can request JSON or XML using Accept header. Spring Boot uses message converters to convert objects into requested format.

In REST design, use path variable for identity and query parameter for optional criteria. This makes APIs readable and predictable.

- Path variable is part of URI path.
- Query parameter comes after question mark.
- Accept header tells desired response type.
- Content-Type header tells request body type.
- Message converters handle JSON/XML conversion.

Feature	Example	Purpose
Path Variable	/students/10	Identify one resource
Query Parameter	/students?name=Rahul	Filter/search
Accept Header	application/json	Requested response format
Content-Type	application/json	Request body format

```
@GetMapping("/students/{id}")
StudentDto get(@PathVariable Long id) { ... }

@GetMapping("/students")
List<StudentDto> search(@RequestParam String name) { ... }
```

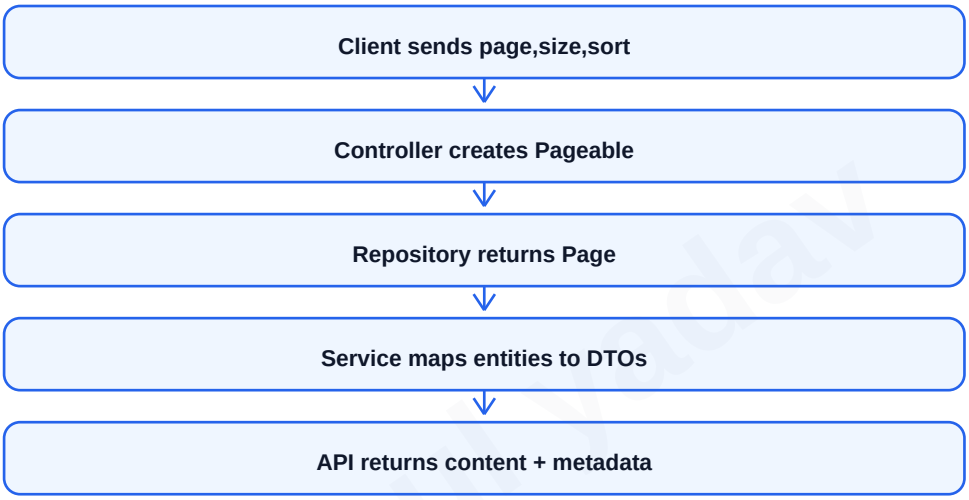
Pagination, Sorting and REST Best Practices

Pagination divides a large result set into smaller pages. It improves performance and avoids sending too much data in one response. Sorting arranges data based on a field such as name, date or id.

Spring Data JPA supports pagination using Pageable and Page objects. Clients can pass page, size and sort parameters. The API response can include content, totalElements, totalPages and current page number.

REST best practices include using correct methods, clear URIs, proper status codes, DTOs, validation, consistent error format, versioning when needed and secure authentication.

- Use pagination for list APIs.
- Limit maximum page size to protect server.
- Use sort=name,asc or sort=id,desc style.
- Return metadata with paged response.
- Document API endpoints clearly.




```
@GetMapping
Page<StudentDto> list(Pageable pageable) {
    return service.findAll(pageable);
}
```

Important Experiments and Practical Programs

The practical list in the syllabus mainly tests basic web flow, servlet/JSP handling, session tracking, error handling, XML configuration and database operations. For practical exams, focus on input form, controller/servlet, output page and database connection steps.

Always write imports, class declaration, mapping annotation/XML mapping, request parameter reading, response content type and output statement. For JSP, separate HTML form and processing JSP when possible.

- HttpServlet demonstration: create servlet, map URL, override doGet, print output.
- HTML to servlet: form action targets servlet URL, servlet reads request.getParameter.
- Basic JSP: use expression/scriptlet or EL to display dynamic data.
- Database in JSP: load driver, create connection, execute query, show result table.
- Session in JSP: session.setAttribute and session.getAttribute.
- Custom tags: create tag handler/TLD or use JSTL tags.
- Error handling: configure errorPage and isErrorPage.
- Auto refresh: response header Refresh or meta refresh.
- XML mapping: define servlet and servlet-mapping in web.xml.
- Relationship mapping: use JPA annotations and repository test.

Experiment	Core Concept	Expected Output
HttpServlet	doGet/doPost	Dynamic HTML response
Form to Servlet	request.getParameter	Display submitted value
JSP Example	JSP lifecycle/elements	Dynamic page
Database in JSP	JDBC	Insert/select records
Session JSP	HttpSession	Login/cart persistence
JPA Mapping	ORM annotations	Related tables created

Quick Revision: Exam Writing Format

For a 10-mark answer, use a fixed structure: definition, need/purpose, architecture or lifecycle diagram, working steps, table/comparison, code snippet and conclusion. This makes the answer complete even when the question is broad.

For short notes, write 5 to 7 crisp points with one example. For differences, always draw a two-column table. For lifecycle questions, draw a flow diagram first and then explain each stage.

Important long-answer questions include servlet lifecycle, session management, JSP implicit objects, IoC and DI, bean lifecycle, Spring Boot auto-configuration, JPA relationships, MVC architecture and REST architecture.

- Servlet lifecycle: init, service, destroy.
- JSP lifecycle: translation, compilation, jspInit, _jspService, jspDestroy.
- Spring core: IoC container manages beans.
- Spring Boot: starters plus auto-configuration.
- JPA: entity mapping and repositories.
- MVC: DispatcherServlet controls request flow.
- REST: resource, URI, HTTP method, representation, statelessness.

Question Type	Best Answer Pattern
Lifecycle	Definition + flow diagram + method explanation
Architecture	Layer diagram + component roles
Difference	Comparison table + examples
Program	Steps + code + output explanation
REST API	URI + method + status code + DTO